PackMonitor: Towards Zero Package Hallucinations via Decoding-Time Monitoring

XITING LIU, Tsinghua University, China
YUETONG LIU, Tsinghua University, China
YITONG ZHANG, Tsinghua University, China
JIA LI, Tsinghua University, China
SHI-MIN HU, Tsinghua University, China

arXiv:2602.20717v2 [cs.SE] 8 Oct 2026

As Large Language Models (LLMs) are increasingly integrated into software development workflows, their trustworthiness has become a critical concern. In dependency recommendation scenarios, the reliability of LLMs is undermined by widespread package hallucinations, where models often recommend hallucinated packages. Recent studies have proposed a range of approaches to mitigate this issue. Nevertheless, existing approaches typically merely reduce hallucination rates rather than eliminate them, leaving persistent software security risks.

In this work, we argue that package hallucinations are theoretically preventable based on the key insight that package validity is decidable through finite and enumerable authoritative package lists. Building on this, we propose *PackMonitor*, the first approach capable of fundamentally eliminating package hallucinations by continuously monitoring the model's decoding process and intervening when necessary. To implement this in practice, *PackMonitor* addresses three key challenges: ❶ determining when to trigger intervention via a Context-Aware Parser that continuously monitors model outputs and selectively activates intervening only during installation command generation; ❷ resolving how to intervene by employing a Package-Name Intervenor that strictly limits the decoding space to an authoritative package list; and ❸ ensuring monitoring efficiency through a DFA-Caching Mechanism that enables scalability to millions of packages with negligible overhead. Extensive experiments on five widely used LLMs demonstrate that *PackMonitor* is a training-free, plug-and-play solution that consistently reduces package hallucination rates to zero while maintaining low-latency inference and preserving original model capabilities.

CCS Concepts: • **Security and privacy** → **Software security engineering**; • **Computing methodologies** → **Natural language processing**; • **Theory of computation** → *Grammars and context-free languages.*

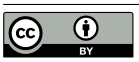
Additional Key Words and Phrases: Package Hallucination, Large Language Models, Constrained Decoding

ACM Reference Format:
Xiting Liu, Yuetong Liu, Yitong Zhang, Jia Li, and Shi-Min Hu. 2026. PackMonitor: Towards Zero Package Hallucinations via Decoding-Time Monitoring. *Proc. ACM Softw. Eng.* 3, ISSTA, Article ISSTA027 (October 2026), 23 pages. https://doi.org/10.1145/3832118

1 Introduction

In recent years, Large Language Models (LLMs) such as Qwen2.5-Coder [20] and DeepSeek-Coder [17], along with LLM-powered agents like Copilot [16] and Cursor [3], have been reshaping

Authors' Contact Information: Xiting Liu, Tsinghua University, Department of Computer Science and Technology, Beijing, China, liuxt25@mails.tsinghua.edu.cn; Yuetong Liu, Tsinghua University, College of AI, Beijing, China, 23371522@buaa.edu.cn; Yitong Zhang, Tsinghua University, College of AI, Beijing, China, 22373337@buaa.edu.cn; Jia Li, Tsinghua University, College of AI, Beijing, China, jia_li@mail.tsinghua.edu.cn; Shi-Min Hu, Tsinghua University, Department of Computer Science and Technology, Beijing, China, shimin@tsinghua.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2994-970X/2026/10-ARTISSTA027
https://doi.org/10.1145/3832118

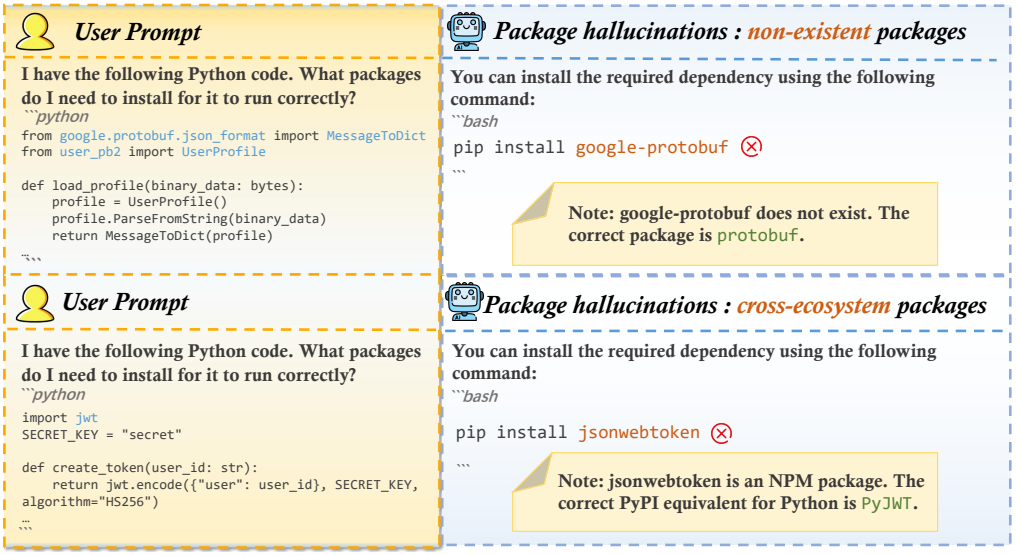


Fig. 1. Examples of active and passive deception in software engineering.

the landscape of software engineering [2, 6, 27, 28, 48, 50]. As these systems become deeply integrated into real-world software development workflows, their trustworthiness has become a critical concern [13, 29, 52, 53]. Unfortunately, current models frequently exhibit hallucinations [13, 30, 49], which substantially undermine trustworthiness and introduce risks to the software lifecycle.

Among the various forms of hallucination, package hallucination has received particular attention [23, 25, 41], where a model may recommend non-existent software packages (top of Figure 1) or suggest packages from the wrong ecosystem (bottom of Figure 1). Prior work shows that package hallucinations are widespread: across 16 widely used LLMs, hallucinated packages can account for up to 19.7% of recommended packages [55], substantially undermining model trustworthiness. We argue that widespread package hallucinations create a concrete attack surface for software supply chain attacks [38]. In practice, LLMs often repeatedly output a small set of hallucinated package names, enabling adversaries to preemptively register them as malicious packages. When such a name is later recommended and mistakenly installed, the malicious dependency can be introduced into the software stack, triggering downstream compromises.

The severity of package hallucinations has motivated a range of approaches aimed at mitigating them [22, 26, 43, 44]. However, we find that existing approaches can only reduce the likelihood of package hallucinations, rather than guarantee their elimination. For example, Retrieval-Augmented Generation (RAG) injects retrieved valid package names into the prompt, yet on DeepSeek-Coder-6.7B [17] it only reduces the hallucination rate from 16.14% to 12.24%. Self-Refinement (SR) asks the model to verify and regenerate its own outputs, but yields only marginal gains (e.g., CodeLlama [40] drops from 26.28% to 25.51%). Supervised fine-tuning (SFT) retrains models on hallucination-free data, but still cannot fully eliminate hallucinations and may also degrade utility. As a result, the remaining package hallucinations continue to pose an exploitable software supply chain threat [24, 38], hindering the broader adoption of LLMs in software development.

To improve the trustworthiness of LLMs, we propose *PackMonitor*, the first approach capable of fundamentally eliminating package hallucinations. Our key insight is that valid package names reside within a finite, enumerable authoritative package list (e.g., the PyPI index [39]) [41]. This makes package-name validity decidable, rendering package hallucinations theoretically preventable.

Building on this insight, we propose *PackMonitor*, which continuously monitoring the model's decoding process and intervening when necessary. Concretely, it intervenes in the decoding phase by pruning any generation path that would inevitably lead to a non-existent package name via logits masking. In this way, hallucinated packages are excluded from the generation space, guaranteeing a zero package-hallucination rate. Notably, *PackMonitor* requires no additional training and operates as a lightweight, plug-and-play solution.

However, implementing *PackMonitor* in practice comes with three key challenges. ❶ First, *PackMonitor* must determine **when to trigger intervention**. This is crucial because security risks stem exclusively from executable package installation commands. Applying intervention globally would inevitably penalize benign text or ordinary code, thereby reducing the model's utility. To address this, *PackMonitor* employs a **Context-Aware Parser** that operates in a continuous monitoring mode but activates strict intervention only when detecting package-name generation following installation commands. This selective mechanism ensures that the model's original generation capabilities remain preserved outside the high-risk scope. ❷ Second, we address **how to intervene LLMs' generation**. To guarantee zero package hallucinations under the enforced package list, *PackMonitor* employs a **Package-Name Intervenor**. Upon activation, the intervenor leverages a Deterministic Finite Automaton (DFA) constructed from an authoritative package list to precisely delimit the valid generation space. At each decoding step, the intervenor derives the exact set of permissible next tokens and rules out any illegal continuations. Consequently, any path leading to a hallucinated package name is strictly removed from the generation space. ❸ Third, we address **how to perform monitoring efficiently**. In practice, the scale of valid packages is massive—for instance, PyPI contains over 700k packages. At this scale, the resulting DFA becomes extremely large, making on-the-fly construction prohibitively slow. To ensure scalability, *PackMonitor* introduces a **DFA-Caching Mechanism**: it pre-constructs the DFA into a persistent checkpoint and loads it into memory during inference. This "build once, reuse many" strategy decouples construction costs from runtime execution, enabling low-latency serving even for massive ecosystems.

We conducted extensive experiments on two benchmarks, including HFuzzer [55] and Package4U [41] for evaluating package hallucinations, using five widely used LLMs and multiple baselines. Our findings are as follows: ❶ Across all experimental settings, *PackMonitor* strictly reduces package hallucination rates to zero. For instance, it successfully eliminate the 11.6% package hallucination rates originally observed in DeepSeek-Coder-6.7B. ❷ *PackMonitor* introduces only negligible inference overhead, with the generation time per response increasing by merely 0.05–0.3 seconds. ❸ Unlike SFT, which can significantly degrade code generation capability, our results on HumanEval [10] indicate that equipping models with *PackMonitor* does not harm their utility.

In summary, our contributions are threefold:

- We prove that package hallucination is a unique type of factuality hallucination that can be theoretically prevented, owing to its deterministic nature, which allows candidate package names to be verified against finite authoritative package lists.
- We propose *PackMonitor*, the first approach capable of completely eliminating package hallucinations. *PackMonitor* integrates a context-aware parser with a package-name intervenor and an efficient DFA-caching mechanism.
- We conduct extensive experiments using two benchmarks on five widely used LLMs. The results show that *PackMonitor* reduces package hallucination rates to zero across all settings while maintaining low-latency inference.

2 Related Work and Motivation

In this section, we present the essential background for understanding our proposed approach and discuss the research motivation behind it.

2.1 LLM Hallucinations and Package Hallucinations

With the rapid development of Large Language Models (LLMs), **LLM hallucinations** [19, 21, 51] have become a widely recognized issue, where LLMs generate misleading, unsubstantiated, or completely fictitious information [13]. LLM hallucinations can be broadly classified into **factuality hallucinations** and **faithfulness hallucinations** [32, 47]. Factuality hallucinations arise when the generated content deviates from verifiable real-world facts, whereas faithfulness hallucinations occur when there are inconsistencies between the generated content and the given input context. [19].

In recent years, LLM-powered coding agents have garnered considerable attention with their rapid development [8, 29, 45], highlighting the immense potential of LLMs in software development. However, the impact of hallucinations in this domain has been further magnified. **Package hallucination** [23, 25, 41] is a specialized form of factuality hallucination, in which LLMs generate non-existent or ecosystem-incompatible package names in the output. Spracklen et al. [41] conducted an evaluation on 16 open-source and closed-source LLMs, generating over 1 million package recommendations from around 40,000 programming questions collected from LLM-generated content and Stack Overflow [42]. Their findings indicated that 19.7% of the recommended packages were hallucinatory, with 205,474 distinct non-existent package names identified. In addition, Zhao et al. [55] proposed HFuzzer, a fuzzing-based framework to evaluate whether models generate hallucinated packages. The experiments using HFuzzer demonstrate that most models, including both open-source and proprietary commercial models, produce a substantial number of hallucinated packages.

In automated coding agent workflows, invalid package dependencies may cause build failures and deployment disruptions, thereby increasing the overhead of development and debugging processes [18]. More dangerously, as discussed in Section 1, package hallucinations introduce a critical attack vector for **software supply chain attacks** [24, 36, 38]. Attackers no longer need to guess or obfuscate popular package names; instead, they can monitor the outputs of mainstream LLMs at scale and collect the frequently generated hallucinated package names for targeted preemptive registration. Once an automated coding agent installs such a maliciously registered package, the attacker's code may be executed within the agent's runtime environment, leading to severe consequences such as data exfiltration, unauthorized network access, and further compromise of downstream systems in the software development pipeline [24, 36, 38] .

2.2 Mitigation of Package Hallucinations

Existing methods for mitigating package hallucinations in LLMs can be broadly categorized by whether they require additional model training, which yields two primary categories: **training-free** and **training-based** methods [41, 44]. The former mitigates hallucinations via prompt-based guidance or post-hoc verification, while the latter relies on parameter-level adjustments to the model.

Training-free Methods. To avoid the computational costs and unintended side effects of retraining, several training-free methods have been proposed to mitigate package hallucinations without modifying model parameters. Two representative techniques are **Retrieval-Augmented Generation (RAG)** [14, 26] and **Self-Refinement (SR)** [22, 31]. RAG mitigates package hallucinations by injecting externally retrieved, valid package information into the prompt, thus enhancing factual grounding during generation. Although this approach can reduce hallucination rates in certain scenarios, its effectiveness is often limited and can vary across settings. Empirical findings demonstrate that RAG only yields modest performance improvements; for example, it reduces the package hallucination rate of DeepSeek-Coder-6.7B from 16.14% to 12.24% [41]. SR, on the other

hand, prompts the LLM to verify and optionally regenerate content following the initial generation step. This method frames hallucination mitigation as a post-hoc correction task, which relies on the model's ability to identify its own factual errors. Although SR can achieve marginal reductions in package hallucination rates, it is inherently unreliable: once an LLM generates hallucinated packages with high confidence, SR often fails to detect or correct such errors. Additionally, SR incurs extra inference overhead, as it requires additional verification and regeneration steps during the inference process. Recent work has also shown that decoding-time monitoring can improve the reliability of code generation. Agrawal et al.use static analysis over repository context to guide Code LM decoding and improve compilation-related correctness properties [1]. Our work is complementary: instead of enforcing repository-level type or API constraints, we focus on preventing invalid package names in generated installation commands.

Training-based Methods. Several studies have mitigated package hallucinations via **Supervised Fine-Tuning (SFT)** [41, 43]. In this paradigm, LLMs are fine-tuned on carefully curated datasets that exclude hallucinated package names.By adjusting the model's parameters, SFT can significantly reduce the model's tendency to generate hallucinated package names. For example, Spracklen et al. [41] demonstrate that SFT can reduce the package hallucination rates to as low as 2.66% for certain LLMs. However, training-based mitigation suffers from several fundamental limitations. First, SFT requires the construction of high-quality, hallucination-free training datasets, which is expensive and difficult to maintain as software ecosystems evolve. Second, altering the model's parameters may introduce unintended side effects; previous studies have shown that SFT can cause a significant degradation in the model's general code generation performance[41]. Most importantly, despite its effectiveness in reducing package hallucination rates, SFT does not fully eliminate package hallucinations under existing settings. Consequently, package hallucination remains possible, which leaves residual security vulnerabilities in the downstream software supply chain.

Summary. Despite their inherent differences, both training-free and training-based methods suffer from a fundamental limitation: they can only mitigate package hallucinations rather than completely eliminate them. Training-free methods rely on prompt-based guidance or post-hoc verification, thus offering no strict generation-time guarantees; training-based methods reduce package hallucinations at the expense of retraining overhead and degraded model performance. As long as the package hallucination rate remains non-zero, such hallucinations can still introduce exploitable security vulnerabilities, which motivates the need for novel methods that can fundamentally prevent the generation of hallucinated packages.

2.3 Constrained Decoding for Code Generation

Constrained decoding [5, 9, 15] has been explored as a way to improve the reliability and controllability of LLM generation. Instead of relying solely on post-hoc verification or repair, constrained decoding incorporates external constraints directly into the decoding process, pruning invalid generation paths before they are sampled. In code generation, such constraints are particularly useful because many correctness properties can be partially specified by formal structures, such as grammars, type systems, static-analysis rules, API specifications, or repository-level program context.

Recent studies have applied constrained decoding to different software engineering tasks. For example, grammar-based or syntax-aware decoding can ensure that generated code follows certain syntactic structures [33]. Other approaches incorporate semantic constraints, such as type constraints or API-usage constraints, to guide model generation toward code that is more likely to compile or match the surrounding program context [12, 46]. Closely related to our work, Agrawal

et al. propose monitor-guided decoding, which uses static analysis over repository context to guide Code LM decoding and avoid invalid continuations, such as unavailable or inconsistent API usages [1]. These studies show that decoding-time constraints can effectively improve code generation reliability by leveraging deterministic information from programming languages and software repositories.

Our work is inspired by this line of research, but focuses on package hallucination, where the validity of generated package names can be checked against a finite and enumerable package registry. We further discuss how this observation motivates the design of *PackMonitor* in the next section.

2.4 Motivation

The constrained decoding perspective provides a natural way to revisit package hallucinations. Package installation in software development exhibits a key characteristic that distinguishes it from many open-ended factuality problems: package-name validity is decidable under a given software ecosystem. For a given package ecosystem, such as PyPI [39], npm [37], or Maven [4], a finite and enumerable list of valid package names can be obtained as ground truth, either from an official registry snapshot or a curated allowlist. Thus, for any candidate package name, its membership in this list is unambiguous: it either belongs to the set or not.

This decidability property allows package hallucination to be formulated as a constrained decoding problem. Consider the hallucinated package example in Figure 1: when prompted to install Protocol Buffers, an LLM may suggest "google-protobuf" due to spurious statistical correlations learned from its training data. While "google.protobuf" is a legitimate Python import path, it is not a valid package name in the predefined authoritative package list. With access to such an allowlist, *PackMonitor* intervenes in real time as the LLM generates a package name token by token. Conceptually, *PackMonitor* intervenes as follows: for each candidate next token, *PackMonitor* checks whether appending the token would keep the current partial string consistent with at least one allowlisted package name (i.e., whether the updated prefix remains a valid prefix for any legitimate package name in the allowlist). If a token would lead the generation process to a state where no allowlisted package name can be matched by any subsequent continuation, selecting the token would inevitably result in an invalid package name; *PackMonitor* thus masks the token by setting its corresponding logit value to $-\infty$. Meanwhile, a termination token (e.g., whitespace or newline) is allowed only if the current prefix already constitutes a complete, allowlisted package name. In this manner, *PackMonitor* prunes all token choices that would inevitably result in a hallucinated package, while preserving all valid package name completions.

However, making this idea practical requires addressing two challenges that are specific to package hallucination. First, package-name constraints should not be applied globally. LLM responses often contain natural language explanations, code snippets, comments, and ordinary shell commands. Enforcing package-name constraints over the entire output would unnecessarily restrict the model and may degrade its general generation capability. Therefore, *PackMonitor* introduces a context-aware parser that continuously monitors the output but activates strict constrained decoding only when package names are being generated in installation commands. Second, package ecosystems are large and continuously evolving, making naive constraint construction inefficient. To address this issue, *PackMonitor* compiles the authoritative package list into a DFA and introduces a caching mechanism to reuse the constructed automaton during inference. In this way, *PackMonitor* adapts the general idea of constrained decoding to the package hallucination problem by formalizing package-name validity as a finite constraint space, applying constraints only in the appropriate generation context, and making the approach efficient enough for large-scale package registries.

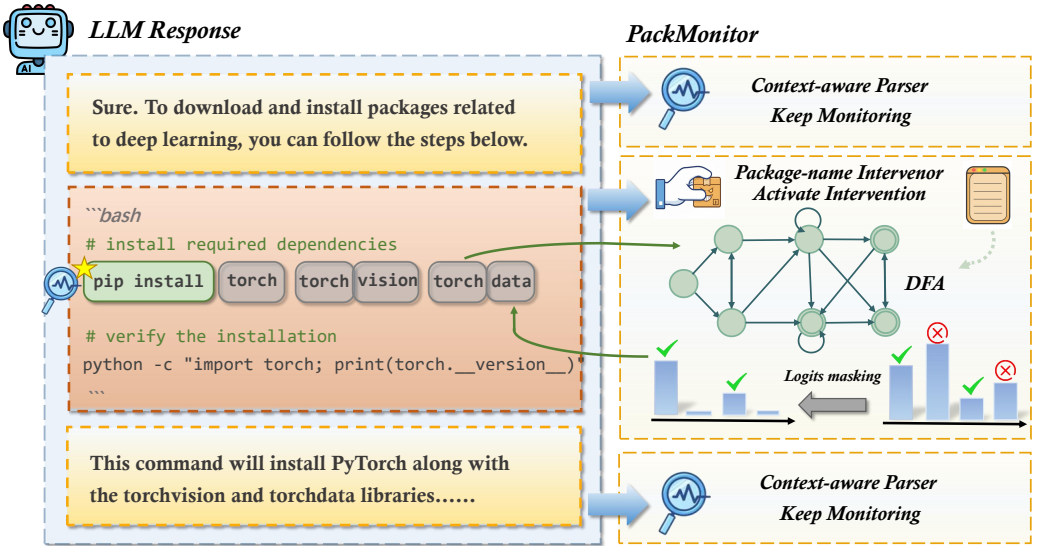


Fig. 2. Overview of our proposed PackMonitor.

3 Methodology

In this section, we present the design of PackMonitor, a decoding-time monitoring framework that is capable of eliminating package hallucination. We begin with a high-level overview of the framework in Section 3.1. We then detail the precise timing of intervention in Section 3.2, the mechanics of the intervention process in Section 3.3, and the optimization strategies for efficient monitoring in Section 3.4.

3.1 Overview

Driven by the insight that package validity is a deterministically decidable problem within a finite and enumerable scope, we propose PackMonitor. The framework orchestrates two tightly coupled components that operate synchronously with LLM inference: the **Context-Aware Parser** and the **Package-Name Intervenor**.

Figure 2 illustrates the overall framework of PackMonitor. First, the context-aware parser operates in a continuous monitoring mode to precisely locate the generation of package names following installation commands. Upon detecting such a context, it triggers the **package-name intervenor**. This module leverages a Deterministic Finite Automaton (DFA), constructed from authoritative package lists, to perform token-level validity verification. It enforces strict compliance by dynamically applying logits masks, thereby blocking any token that would lead to invalid package names.

To tackle the scalability challenge posed by modern ecosystems containing millions of packages, we further introduce a **DFA-Caching Mechanism**. This mechanism decouples the computationally intensive DFA construction from the runtime inference. By pre-compiling the DFA offline and loading the checkpoint during inference, PackMonitor effectively amortizes the initialization cost, enabling it to scale robustly to massive ecosystems with negligible latency overhead.

3.2 When to Trigger Intervention?

In this section, we address the first challenge: determining the precise timing for intervention. A naive approach that intervenes the entire output stream against a package list is infeasible;

it would inevitably penalize benign natural language tokens or generic code, thereby severely degrading the model's general utility. We argue that supply chain security risks stem exclusively from executable package installation commands. Therefore, while monitoring must be continuous, strict intervention must be confined to these high-risk regions to ensure that the model's original generation capability remains untouched.

To achieve this, *PackMonitor* employs a **context-aware parser** modeled on Context-Free Grammar (CFG). This parser functions as a real-time sentinel, continuously monitoring generated tokens to distinguish between risk-free contexts and package-name generation zones.

Formally, we model the response of an LLM as an interleaving sequence of natural language and code segments:

$$\texttt{Response} \rightarrow (\texttt{NATURAL_LANGUAGE} \mid \texttt{CODE})^*. \tag{1}$$

Code blocks are identified by standard delimiters (e.g., ```` ```bash ```` and ```` ``` ````). Within these code blocks, the parser further differentiates between actual installation commands—which constitute the attack surface that we focus on—and ordinary code lines (such as comments or variable assignments):

$$\texttt{CODE} \rightarrow (\texttt{INSTALL_CMD_LINE} \mid \texttt{ORDINARY_CODE_LINE})^*. \tag{2}$$

While we use the prevalent `pip install` command for PyPI packages as a running example, our approach is general and can be instantiated for installation commands in other ecosystems (discussed in Section 6.2). Under the PyPI ecosystems, an installation command line is formally defined as:

$$\texttt{INSTALL_CMD_LINE} \rightarrow \texttt{pip install PACKAGE_NAMES \textbackslash n}. \tag{3}$$

Crucially, since LLMs frequently recommend multiple packages in a single command, the parser must support iterative validation. Once an installation command is recognized, *PackMonitor* enforces intervention for every subsequent token sequence identified as a package name:

$$\texttt{PACKAGE_NAMES} \rightarrow \texttt{PACKAGE_NAME}\ (\texttt{PACKAGE_NAME})^*. \tag{4}$$

During inference, the context-aware parser operates in a continuous monitoring mode. As long as the output state corresponds to `NATURAL_LANGUAGE` or `ORDINARY_CODE_LINE`, the package-name intervenor remains dormant, incurring zero interference. However, the moment the parser detects a state transition into `PACKAGE_NAMES`, it immediately triggers active intervention. This design effectively shifts *PackMonitor* from passive context-tracking to strict validity enforcement, ensuring zero package hallucinations without compromising the generation of benign text.

3.3 How to Intervene in LLM Generation?

Once activated, the **package-name intervenor** assumes strict control over the generation process. Unlike post-hoc verification methods, this module operates synchronously during the decoding time. It functions in two coordinated stages: (i) it rigorously defines the legal generation space using a Deterministic Finite Automaton (DFA); and (ii) it performs intervention on the probability distribution via logits masking to render hallucinated packages theoretically unreachable.

3.3.1 Legal State Verification via DFA. To guarantee that every generated token contributes to a valid package name, *PackMonitor* compiles the authoritative package list into a DFA.

DFA Construction. We formalize the package validity specification as a DFA $D = (Q, \Sigma, \delta, s_0, F)$, where Q is the state space, Σ is the character alphabet, $\delta : Q \times \Sigma \rightarrow Q$ is the deterministic transition function, $s_0 \in Q$ is the initial state, and $F \subseteq Q$ is the set of accepting states. By treating each package name p_i in the authoritative package list $P = \{p_1, \ldots, p_N\}$ as a terminal string, the DFA serves as a

deterministic realization of the following production rule:

$$\texttt{PACKAGE_NAME} \rightarrow p_1 \mid p_2 \mid \cdots \mid p_N. \tag{5}$$

During inference, *PackMonitor* keeps track of the DFA state induced by the package-name prefix generated so far. When the model outputs the next token, *PackMonitor* interprets it as a character string c_t and updates the state by consuming these characters sequentially:

$$s_t = \delta^*(s_{t-1}, c_t). \tag{6}$$

where δ^* denotes the extended transition function over a character sequence.

Token Trie Integration. The necessity of integrating a Token Trie arises fundamentally from the granularity mismatch between the generation unit of LLMs and the verification unit of the DFA. LLMs operate on a vocabulary V of subword tokens, generating a multi-character string v at each step. In contrast, the constructed DFA defines validity via character-level transitions $\delta : Q \times \Sigma \rightarrow Q$. Consequently, a single generation step by the LLM corresponds to a sequence of state transitions in the DFA, making direct verification impossible without decomposing the token.

To bridge this gap, we pre-build a **Token Trie** from the tokenizer's vocabulary. This Trie structurally maps each token $v \in V$ to its constituent character sequence. By synchronizing the traversal of the Token Trie with the DFA, we can determine whether the character sequence of a candidate token v forms a valid path starting from the current DFA state s_t. Formally, a token v is included in the feasible set $A(s_t)$ if and only if its character sequence allows the DFA to transition from s_t to a valid subsequent state (i.e., not the empty set).

3.3.2 Logits-Level Masking. Once the set of feasible tokens $A(s_t)$ is identified, *PackMonitor* (which is denoted as P below) enforces compliance at the probabilistic level. Specifically, we apply a logits mask that strictly forbids the generation of any token outside the legal set.

In standard autoregressive decoding, the LLM L_θ computes the next-token probability based on logits ℓ_t:

$$L_\theta(x_t \mid x_{<t}) = \text{softmax}(\ell_t). \tag{7}$$

We model the intervened system as a reshaped distribution $L_\theta \parallel P$, where the original logits are modulated by a dynamic mask m_t derived from $A(s_t)$:

$$(L_\theta \parallel P)(x_t \mid x_{<t}) = \text{softmax}(\ell_t \oplus m_t). \tag{8}$$

The mask m_t acts as a binary gatekeeper over the vocabulary V:

$$m_t(v) = \begin{cases} 1, & v \in A(s_t), \\ 0, & v \notin A(s_t). \end{cases} \tag{9}$$

The operator $\oplus$ applies a mask to the logits:

$$(\ell_t \oplus m_t)[v] = \begin{cases} \ell_t[v], & \text{if } m_t(v) = 1, \\ -\infty, & \text{if } m_t(v) = 0. \end{cases} \tag{10}$$

By reshaping the distribution L_θ, our proposed package-name intervenor ensures that the probability of sampling an invalid token becomes exactly zero. Consequently, every generated prefix is guaranteed to remain on some valid DFA path; if decoding terminates in an accepting state, the produced package name is guaranteed to be a valid entry in the authoritative package list.

3.4 How to Efficiently Perform Monitoring?

In practical scenarios, the DFA constructed from large-scale package lists exhibits an extremely large state space, leading to substantial overhead in each construction. Our extensive experimental results indicate that constructing the DFA for a large-scale package list is the primary bottleneck of inference latency. For instance, with the PyPI package list containing approximately 700k package names, constructing the DFA takes several seconds, whereas performing token-level validation using the Token Trie requires only a few milliseconds or even microseconds.

To efficiently perform monitoring in practical settings, we introduce a **DFA-caching mechanism** that decouples one-time automaton construction from runtime inference. Our key insight is that, for a fixed package list, the structure of the DFA remains unchanged during inference. Leveraging this property, *PackMonitor* pre-constructs a DFA checkpoint during initialization and stores it in cache. For each generation request, *PackMonitor* loads the cached DFA directly, thereby avoiding repeated construction.

From a runtime perspective, this DFA-caching mechanism ensures that the expensive cost of initializing the DFA is incurred only once. This "build once, reuse many" design effectively decouples inference latency from the size of the package list, leaving the online overhead dominated by lightweight DFA state transitions and logits-level masking operations, both of which are computationally stable and negligible during inference. This mechanism enables *PackMonitor* to operate efficiently in million-scale software ecosystems.

4 Experimental Setup

This section describes our experimental setup, including the benchmarks, baseline methods, evaluation metrics, selected models, and implementation details. Under this setup, we conduct a series of experiments to answer the following Research Questions (RQs):

- **RQ1: How effective is *PackMonitor* in eliminating package hallucinations?**
  This RQ evaluates whether *PackMonitor* can eliminate package hallucinations across different models and compares its performance against existing baselines.
- **RQ2: What is the efficiency overhead introduced by *PackMonitor*?**
  This RQ measures the efficiency impact of *PackMonitor* by comparing the average generation speed (or latency) before and after integrating *PackMonitor*.
- **RQ3: Does *PackMonitor* degrade the general code generation capability of LLMs?**
  This RQ examines whether *PackMonitor* affects the general code generation capability of LLMs.
- **RQ4: How much does DFA-caching mechanism improve inference efficiency and scalability?**
  This RQ is studied via an ablation analysis comparing cached versus non-cached settings, and further characterizing how inference latency scales with increasing package-list sizes.

4.1 Benchmarks

To evaluate package hallucinations in LLMs, we adopt two benchmarks in our experiments: the **HFuzzer** framework [55] and an open-source coding dataset released by Spracklen et al. [41].

❶ **HFuzzer [55].** We adopt HFuzzer as our primary evaluation framework. HFuzzer is a fuzzing framework designed to systematically probe failure modes of LLMs through iterative test case generation. Compared to conventional fixed test cases, HFuzzer can continuously explore diverse generation patterns and effectively induce previously unseen hallucinated packages. In the experiments, HFuzzer involves two cooperating models: a **tester model** for test case generation, and a **target model** responsible for code generation and package recommendation, serving as the model

Table 1. The Details of Selected Models.

Model	Size	Release Date	Download Count
DeepSeek-Coder-Instruct-6.7B [17]	6.7B	2023-11	56K
CodeLlama-7b-Instruct-hf [40]	7B	2023-08	19K
Qwen2.5-Coder-7B-Instruct [20]	7B	2024-09	1.18M
Meta-Llama-3-8B-Instruct [34]	8B	2024-04	1.53M
Meta-Llama-3.1-8B-Instruct [35]	8B	2024-07	10M

under evaluation. We select **DeepSeek-V3** [11] as the tester model, as its strong reasoning and coding capabilities support effective and diverse code-relevant test case generation.

❷ **Package4U [41].** In addition to HFuzzer, we also evaluate our approach on a dataset released by Spracklen et al. [41]. Following the title of the original paper, we refer to this dataset as **Package4U**. Package4U contains approximately 40,000 coding tasks generated by LLMs or collected from Stack Overflow, with around half involving Python programming. Each task consists of a natural language problem description and code-related context. We use this dataset in a subset of our experiments. Detailed experimental settings and results are reported in the corresponding sections.

4.2 Models

We evaluate our approach on a set of representative LLMs, as summarized in Table 1. Since *PackMonitor* is model-agnostic and can be applied to different LLMs without modifying their parameters, we follow the model selection of prior work, i.e., HFuzzer [55], to ensure a fair comparison. The selected models cover different training objectives, including code-oriented and general-purpose LLMs, as well as different release periods.

- DeepSeek-Coder-Instruct-6.7B [17] is released by DeepSeek. It is an instruction-tuned, code-oriented LLM designed for programming tasks. In the following, we refer to this model as DeepSeek-Coder.
- CodeLlama-7b-Instruct-hf [40] is released by Meta. It is an instruction-tuned code model in the Code Llama family, widely adopted as a strong open-weight baseline for code generation. In the following, we refer to this model as CodeLlama.
- Qwen2.5-Coder-7B-Instruct [20] is released by the Qwen team (Alibaba). It is an instruction-tuned code model from the Qwen2.5 series, optimized for coding and tool-use style interactions. In the following, we refer to this model as Qwen2.5-Coder.
- Meta-Llama-3-8B-Instruct [34] is released by Meta. It is a general-purpose instruction-tuned LLM that provides a strong non-code-specialized baseline for evaluating robustness beyond code-centric training. In the following, we refer to this model as Meta-Llama-3.
- Meta-Llama-3.1-8B-Instruct [35] is released by Meta as an updated general-purpose instruction-tuned LLM. Compared with Llama-3-8B-Instruct, it improves overall instruction-following and capability. In the following, we refer to this model as Meta-Llama-3.1.

4.3 Baselines

Following prior work on mitigating package hallucinations [41], we include three representative baselines in our evaluation: **Retrieval-Augmented Generation (RAG)**, **Self-Refinement (SR)**, and **Supervised Fine-Tuning (SFT)**. Their details are introduced in Section 2.2. we additionally include a stronger baseline, **RAG+SR**, which combines Retrieval-Augmented Generation with post-hoc Self-Refinement: we first apply RAG to inject retrieved package information into the prompt, and then run SR to verify and refine the model's generated installation commands.

4.4 Evaluation Metrics

To evaluate the effectiveness of PackMonitor and the compared baselines, we adopt three metrics proposed by prior work [41, 55] that quantify hallucinated packages in model outputs, as described below:

- **Package Hallucination Rate (PHR)**. This metric is defined as the ratio of hallucinated packages to the total number of recommended packages.
- **Response Hallucination Rate (RHR)**. This metric measures the percentage of model responses that contain at least one hallucinated package.
- **Number of Unique Hallucinated Packages ($P_{hall-unique}$)**. This metric is analogous to a commonly used indicator in prior work [54], i.e., the number of unique bugs, and reflects the diversity of hallucinated packages generated by the model.

4.5 Implementation Details

For all experiments, we use the PyPI [39] index as the authoritative package list for DFA construction. This list comprises 706,618 valid package names at the time of our experiments, serving as the source for monitoring package-name generation.

For each target model listed in Table 1, we execute one full round of HFuzzer testing (with each round consisting of 1,000 iterations) with PackMonitor and all baselines. To ensure the determinism of experimental results, the temperature parameter of the tester model is fixed at 0, while that of the target model is set to 0.7 to strike a balance between generation creativity and output stability [7].

All experiments are conducted on a server equipped with eight NVIDIA A100 GPUs with 80 GB memory each, two AMD EPYC 7543 CPUs with 32 cores per socket (64 physical cores in total), 1 TB system memory, running Ubuntu 22.04 LTS.

5 Experimental Results

5.1 RQ1: How Effective is PackMonitor in Eliminating Package Hallucinations?

In this RQ, we compare PackMonitor with representative training-free and training-based approaches for mitigating package hallucinations.

5.1.1 *Comparison with training-free methods.*

Settings. We evaluate PackMonitor and the training-free baselines introduced in Section 4.3 on the five LLMs described in Section 4.2, using HFuzzer. We report the main metrics—PHR, RHR, and $P_{hall\text{-}unique}$—as defined in Section 4.4. In addition, we include two auxiliary statistics: P_{total}, the total number of packages recommended in each HFuzzer round, and P_{hall}, the total number of hallucinated packages in each HFuzzer round.

Results. The results of different approaches are reported in Table 2.

***PackMonitor* consistently outperforms all training-free baselines across every model, reducing all hallucination-related metrics to absolute zero.** As shown in Table 2, PackMonitor eliminates package hallucinations on all five LLMs: for example, on DeepSeek-Coder, the vanilla setting exhibits a PHR/RHR of 8.39%/11.60%, while PackMonitor achieves 0% PHR/RHR. Similar results hold for Meta-Llama-3.1 and Qwen2.5-Coder, where PackMonitor consistently drives all hallucination metrics to zero. These results indicate that PackMonitor provides a strict guarantee of eliminating package hallucinations and generalizes across model families. In contrast, existing mitigation methods exhibit limited and sometimes unstable effectiveness. RAG can even *increase* hallucination rates in several settings—for instance, on Qwen2.5-Coder, PHR increases from 3.25% (vanilla) to 3.33% (RAG), and on Meta-Llama-3, RHR increases from 6.40% to 7.10%—which we

Table 2. Comparative performance of *PackMonitor* against training-free baselines on the HFuzzer benchmark. P_{total} denotes the total number of recommended packages, while P_{hall} and $P_{hall-unique}$ represent the total and unique hallucinated packages, respectively. PHR and RHR indicate the package and response hallucination rates.

Model	Configuration	P_{total}	P_{hall}	$P_{hall\text{-}uniq}$	PHR	RHR
DeepSeek-Coder	vanilla	1847	155	74	8.39%	11.60%
	RAG	2288	132	87	5.77%	10.10%
	SR	1496	113	47	7.55%	8.20%
	RAG + SR	1750	108	46	6.17%	8.90%
	PackMonitor	**1648**	**0**	**0**	**0%**	**0%**
CodeLlama	vanilla	1004	63	63	6.27%	2.70%
	RAG	690	41	39	5.94%	2.70%
	SR	502	20	20	3.98%	1.70%
	RAG + SR	231	3	3	1.30%	0.30%
	PackMonitor	**1514**	**0**	**0**	**0%**	**0%**
Qwen2.5-Coder	vanilla	553	18	11	3.25%	1.80%
	RAG	600	20	13	3.33%	1.70%
	SR	513	10	8	1.95%	0.90%
	RAG + SR	551	14	13	2.54%	1.40%
	PackMonitor	**529**	**0**	**0**	**0%**	**0%**
Meta-Llama-3	vanilla	1623	80	38	4.93%	6.40%
	RAG	1773	85	56	4.79%	7.10%
	SR	1604	63	39	3.93%	5.10%
	RAG + SR	1775	77	50	4.34%	6.20%
	PackMonitor	**1526**	**0**	**0**	**0%**	**0%**
Meta-Llama-3.1	vanilla	1979	113	88	5.71%	8.20%
	RAG	2116	109	75	5.15%	9.00%
	SR	1848	78	51	4.22%	7.00%
	RAG + SR	2099	100	74	4.76%	8.90%
	PackMonitor	**1938**	**0**	**0**	**0%**	**0%**

hypothesize may be due to prompt-injected information inducing overconfidence and encouraging unsupported outputs. SR yields more consistent reductions, but its hallucination rates remain strictly non-zero. For example, on DeepSeek-Coder it still leaves 113 hallucinated packages (PHR 7.55%, RHR 8.20%).

***PackMonitor* does not significantly affect the number of recommended packages.** Since *PackMonitor* operates by pruning all generation paths that would lead to hallucinated package names, a natural concern is that it may substantially reduce the number of package recommendations produced by the model. To examine this, we report P_{total}, the total number of recommended packages in each fuzzing round. We find that across all models, the P_{total} achieved by *PackMonitor* is comparable to that of the vanilla model. This indicates that *PackMonitor* does not eliminate hallucinations by suppressing package recommendations; instead, it removes hallucinated packages while preserving the model's tendency to recommend valid packages.

5.1.2 *Comparison with training-based method.*

Settings. Spracklen et al. [41] mitigate package hallucinations by performing SFT on CodeLlama and DeepSeek-Coder using hallucination-free data. To faithfully reproduce their setting, we directly

Table 3. Package Hallucination Rates (PHR) comparison between *PackMonitor* and the SFT baseline.

	DeepSeek-Coder	CodeLlama
Vanilla	16.14%	26.28%
SFT	2.66%	10.27%
PackMonitor	**0%**	**0%**

Table 4. Average generation time (seconds) across different LLMs. For each model, the best result is highlighted in bold. Numbers in parentheses indicate the relative inference time normalized by the corresponding vanilla model.

Method	DeepSeek-Coder	CodeLlama	Qwen2.5-Coder	Llama-3	Llama-3.1
Vanilla	1.9809	2.4296	0.8019	0.9020	1.5277
RAG	2.4751 (×1.25)	3.9064 (×1.61)	0.8797 (×1.10)	1.5359 (×1.70)	1.8978 (×1.24)
SR	3.8439 (×1.94)	3.3827 (×1.39)	1.4026 (×1.75)	**1.0031** (×1.11)	**1.6115** (×1.05)
RAG+SR	5.2509 (×2.65)	4.4491 (×1.83)	1.0595 (×1.32)	1.3596 (×1.51)	2.1525 (×1.41)
Ours	**2.2351** (**×1.13**)	**3.0130** (**×1.24**)	**0.8596** (**×1.07**)	1.1515 (**×1.28**)	1.6464 (**×1.08**)

use the fine-tuned checkpoints released by the authors for evaluation. However, we observe that these SFT models exhibit noticeably degraded instruction-following behavior and often fail to consistently comply with the strict output format required by HFuzzer, making them incompatible with the HFuzzer evaluation pipeline. Therefore, following Spracklen et al. [41], we evaluate training-based methods on the Package4U benchmark and report the corresponding package hallucination rates.

Results. The results are shown in Table 3.

Despite requiring no additional training or parameter updates, *PackMonitor* offers a fundamentally stronger guarantee of package-hallucination elimination than training-based approaches. As shown in Table 3, while SFT substantially reduces hallucination rates, it does not eliminate them: non-zero hallucination rates of 2.66% on DeepSeek-Coder and 10.27% on CodeLlama remain, which still pose non-negligible security risks. In contrast, *PackMonitor* consistently reduces the package hallucination rate to zero across all experimental configurations. These results demonstrate that, even compared with training-based methods that require costly fine-tuning pipelines, *PackMonitor* not only avoids training overhead but also provides a more robust and deterministic solution for hallucination elimination.

Answer to RQ1. *PackMonitor* can stably eliminate package hallucinations, whereas existing mitigation approaches can only reduce package hallucinations and cannot eradicate them.

5.2 RQ2: What is the Efficiency Overhead Introduced by *PackMonitor*?

Although *PackMonitor* eliminates package hallucinations, it must remain lightweight to be practical in real deployments. Therefore, we quantify the overhead introduced by *PackMonitor* and compare it against existing approaches.

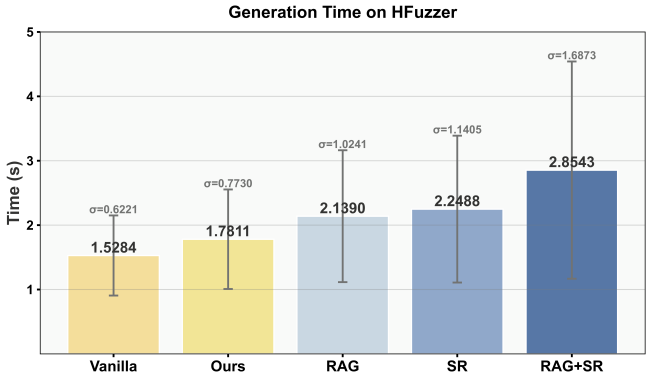


Fig. 3. Average generation time and standard deviation of different methods on HFuzzer, aggregated across LLMs.

Settings. We measure efficiency on HFuzzer. For each HFuzzer round, we evaluate the five models introduced in Section 4.2 with *PackMonitor* and all the training-free baselines introduced in Section 4.3, and report the average generation time (in seconds).

Results. The results are summarized in Table 4 and visualized in Figure 3.

The average overhead is small. Table 4 shows that integrating *PackMonitor* introduces only a modest and consistent slowdown across all five models, with a normalized inference-time ratio between **×1.07 and ×1.28**. In absolute terms, the added latency is limited: +0.0577s on Qwen2.5-Coder (0.8019s → 0.8596s, ×1.07), +0.1187s on Llama-3.1 (1.5277s → 1.6464s, ×1.08), +0.2495s on Llama-3 (0.9020s → 1.1515s, ×1.28), +0.2542s on DeepSeek-Coder (1.9809s → 2.2351s, ×1.13), and ×1.24 on CodeLlama (2.4296s → 3.0130s). Although the relative overhead can reach ×1.28, the absolute latency increase is small, around 0.2s in the worst case. Moreover, *PackMonitor* is among the most time-efficient methods overall: it achieves the best latency on DeepSeek-Coder, Qwen2.5-Coder, and CodeLlama, and the second-best latency on Llama-3.1 and Llama-3. In contrast, RAG and SR often incur substantially higher overhead, and RAG+SR can be particularly expensive (e.g., up to ×2.65 on DeepSeek-Coder).

The latency variance is small and stable. Figure 3 further reports the average generation time and standard deviation aggregated across models. Compared with SR and RAG+SR, *PackMonitor* exhibits noticeably smaller variance, indicating that its overhead is more predictable and does not suffer from occasional latency spikes. This stability is expected because *PackMonitor* performs only lightweight per-step operations (DFA state transitions and logits masking) during decoding, whereas SR and RAG+SR may introduce variable amounts of additional computation due to refinement iterations (and RAG also adds retrieval-related variability).

Answer to RQ2. On HFuzzer, *PackMonitor* incurs only a small and stable efficiency overhead, whereas existing training-free baselines introduce noticeably higher and more variable time costs.

Table 5. Comparison of code generation quality (Pass@1) on HumanEval. Values in parentheses represent the performance drop relative to the vanilla model.

Model	Vanilla	With *PackMonitor*	With SFT
DeepSeek-Coder-6.7B	72.6%	72.6% (↓ 0%)	45.7% (↓ 26.9%)
CodeLlama-7B	18.3%	18.3% (↓ 0%)	4.3% (↓ 14.0%)
Qwen2.5-Coder-7B	79.3%	79.3% (↓ 0%)	-
Meta-Llama-3-8B	53.0%	53.0% (↓ 0%)	-
Meta-Llama-3.1-8B	60.4%	60.4% (↓ 0%)	-

5.3 RQ3: Does *PackMonitor* Degrade the General Code Generation Capability of LLMs?

It is crucial to verify that our proposed *PackMonitor* does not compromise the model's general code generation capabilities. To this end, we evaluate whether *PackMonitor* affects functional correctness on a standard code generation benchmark.

Settings. Following the HumanEval [10] protocol adopted by Spracklen et al. [41], we evaluate the same models before and after integrating *PackMonitor*, and report Pass@1 as the metric for functional correctness. For stability, we set the decoding temperature to 0 for all runs in this RQ.

Results. The results are shown in Table 5.

Across all evaluated models, HumanEval performance remains unchanged after incorporating *PackMonitor*. As shown in Table 5, the Pass@1 scores remain unchanged with and without *PackMonitor*, indicating that our approach does not degrade general code generation capability. We attribute this to the fact that *PackMonitor* only intervenes when the model is generating package names following installation commands. As a result, *PackMonitor* eliminates hallucinated packages without interfering with ordinary code generation, thereby preserving the original utility.

The utility preservation of *PackMonitor* contrasts sharply with SFT. Prior work reports that although SFT can significantly reduce package hallucinations, it may noticeably hurt general code generation performance. On HumanEval, the Pass@1 scores of DeepSeek-Coder-6.7B and CodeLlama-7B drop by 26.9% and 14.0%, respectively [41]. This degradation is expected because SFT changes model weights and may inadvertently harm general code generation performance.

Answer to RQ3. On HumanEval, integrating *PackMonitor* does not change Pass@1 across all evaluated models, indicating that *PackMonitor* preserves general code generation capability.

5.4 RQ4: How Much does DFA-Caching Mechanism Improve Inference Efficiency and Scalability?

In this RQ, we conduct an ablation study to evaluate the effectiveness of the DFA-caching mechanism and to examine how decoding latency scales with increasing package list sizes.

5.4.1 Impact of DFA-Caching on Inference Latency.

Settings. We evaluate the inference time of five LLMs introduced in Section 4.2 under the HFuzzer benchmark across three configurations: the original model, *PackMonitor* integrated without caching (w/o cache), and the full *PackMonitor* with caching enabled.

Results. Experimental results are demonstrated in Table 6.

Table 6. Comparison of average generation time on HFuzzer with and without caching across LLMs. Values in parentheses represent the time multiplier relative to the vanilla model.

Model	Average Generation Time (s)
DeepSeek-Coder-6.7B	1.9809
+ PackMonitor (w/o cache)	5.9150 (×2.99)
+ PackMonitor	2.2351 (×1.13)
CodeLlama-7B	2.4296
+ PackMonitor (w/o cache)	6.8237 (×2.81)
+ PackMonitor	3.0130 (×1.24)
Qwen2.5-Coder-7B	0.8019
+ PackMonitor (w/o cache)	5.0078 (×6.24)
+ PackMonitor	0.8596 (×1.07)
Meta-Llama-3-8B	0.9020
+ PackMonitor (w/o cache)	5.3106 (×5.89)
+ PackMonitor	1.1515 (×1.28)
Meta-Llama-3.1-8B	1.5277
+ PackMonitor (w/o cache)	6.1537 (×4.03)
+ PackMonitor	1.6464 (×1.08)

Table 7. Efficiency gains from parser caching: comparison between one-time offline construction and rapid runtime loading across multi-million package scales.

Time Cost	3000k	700k	70k	7k
DFA Construction	21.121s	4.407s	0.265s	0.021s
DFA Loading	0.247s	0.083s	0.020s	0.002s

Overall, enabling DFA-caching substantially improves decoding efficiency, bringing inference time close to that of the original model. As shown in Table 6, disabling caching incurs a large and consistent overhead across all models (typically ×**2.8**–×**6.2** relative to vanilla), because the DFA must be repeatedly constructed and initialized during online inference. With caching enabled, latency drops sharply and remains close to the vanilla baseline, reducing the overhead to only ×**1.07**–×**1.28**. This gap is particularly pronounced for faster base models (e.g., Qwen2.5-Coder and Llama-3), where on-the-fly DFA construction dominates end-to-end decoding time. Overall, DFA-caching removes the one-time, high-cost automaton construction from the online path, leaving PackMonitor 's runtime overhead dominated by lightweight DFA state transitions and logits masking.

5.4.2 Efficient Scaling to Million-Level Package List Size.

We further evaluate the scalability of PackMonitor by measuring the time overhead of two critical stages in the initialization of the monitoring mechanism: DFA construction and DFA loading.

As shown in Table 7, DFA construction exhibits strong sensitivity to scale. When the package list grows from 7,000 to 3,000,000 entries (approximately a 428× increase), the construction time increases starkly from 0.021s to 21.121s (about a 1000× slowdown). This behavior stems from the rapid expansion of the DFA state space induced by millions of distinct package names. In contrast, DFA loading time is largely insensitive to scale, remaining as low as 0.247s even at the 3,000k scale.

This disparity clearly demonstrates the necessity of DFA-caching mechanism, which converts the substantial compilation cost into a one-time offline overhead.

Without the DFA-caching mechanism, *PackMonitor* would be required to reconstruct the DFA online for every inference request. At a future scale where the package list reaches 3,000k entries, this would incur a construction cost of up to 21.121s per inference—an overhead that is clearly unacceptable for latency-sensitive LLM inference workloads. By introducing DFA-caching, we shift the expensive DFA construction entirely to an offline phase, leaving only a lightweight loading operation at runtime. Empirically, even at the 3,000k scale, DFA loading takes only 0.247s, achieving an approximately 85× speedup compared to construction time costs. This "build once, reuse many" strategy effectively decouples inference latency from package list scale, enabling *PackMonitor* to support million-scale, continuously evolving ecosystems such as PyPI without incurring perceptible overhead during online inference.

Answer to RQ4. Overall, these results demonstrate the effectiveness of the DFA caching mechanism in substantially reducing inference latency. Moreover, the consistently low DFA loading cost across large package lists confirms that *PackMonitor* scales to real-world, evolving software ecosystems.

6 Discussion

In this section, we discuss several key aspects of *PackMonitor* beyond the quantitative results. We first analyze why *PackMonitor* is able to achieve zero package hallucination under our evaluated settings, and clarify the scope of this guarantee. We then examine its generalizability across package ecosystems, highlighting how the framework can be extended beyond the PyPI setting to support diverse installation commands and customizable package lists. Finally, we systematically analyze potential threats to the validity of our study.

6.1 Understanding Why Zero Package Hallucination Is Achievable

A central result of our evaluation is that *PackMonitor* consistently reduces package hallucination rates to zero across all tested models and settings. This outcome may appear unusually strong, especially given that prior work has shown hallucinations to be pervasive and difficult to eliminate. We discuss the key reasons why this guarantee is achievable in our setting.

First, package-name generation fundamentally differs from many other hallucination-prone generation tasks. Unlike open-ended natural language, valid package names form a finite and enumerable list. This property enables *PackMonitor* to derive an exact validity specification and to structurally exclude invalid package names from the generation space, rather than attempting to correct hallucinations after they occur.

Second, *PackMonitor* enforces validity directly at the decoding level, rather than relying on prompt engineering, self-verification, or model retraining. By applying logits-level masking based on an automaton constructed from an authoritative package list, *PackMonitor* guarantees that every generated package-name prefix remains consistent with at least one valid package entry. As a result, hallucinated packages are not merely discouraged but rendered unreachable by construction.

It is important to emphasize that this guarantee is conditional on the correctness and completeness of the given authoritative package list. Under these assumptions, *PackMonitor* ensures that any generated package name is valid according to the repository at generation time.

6.2 Generalization Across Package Ecosystems

Our approach is not limited to the PyPI setting (i.e., monitoring package names only after `pip install`). Owing to its context-aware parser design, *PackMonitor* can be readily adapted, in principle, to different software ecosystems by applying ecosystem-specific package lists to installation commands such as `conda install` and `npm install`. In addition, the set of legitimate packages is not fixed and can be manually imported and customized to accommodate specific environments or private dependency scenarios. This endows *PackMonitor* with robust generalizability and practical deployment flexibility across various package ecosystems.

6.3 Threats to Validity

We now discuss potential threats to the validity of our study and how we mitigate them:

❶ **Generalizability Across Models.** One potential threat concerns whether our findings generalize beyond the specific models. To reduce this threat, we conducted experiments on five widely used LLMs with diverse architectures, training data, and instruction-tuning strategies. In addition, we evaluated *PackMonitor* using HFuzzer, which generates a broad and adversarial set of dependency-related prompts. These choices increase confidence that our results are not artifacts of a particular model or prompt distribution.

❷ **Dependence on Authoritative Package Lists.** *PackMonitor* relies on an externally maintained authoritative package list as the ground truth for package-name intervenor. Any errors or omissions in this list (e.g., misrecorded entries or missing packages) could undermine the correctness of *PackMonitor*. In our experiments, we obtained the package list via PyPI's official API. However, package ecosystems evolve continuously: new packages are published and existing ones may be deprecated over time. To maintain long-term effectiveness, a practical deployment can incorporate a periodic update mechanism to refresh the authoritative list snapshot and synchronize it with the latest ecosystem state, thereby preserving the hallucination-prevention guarantee.

❸ **Impact of Deployment Environments.** For LLMs, generation speed is a critical metric. In our experiments, we evaluated a range of speed-related indicators (e.g., model generation latency, DFA construction time). These metrics typically fall within the millisecond-to-second range, making them potentially sensitive to variations in deployment environments (e.g., differences in GPU or CPU specifications). However, this threat is mitigated by the fact that our core effectiveness (e.g., hallucination-prevention) is independent of such environmental factors. To further support reproducibility across environments, we have made detailed experimental settings available in the Supplementary Materials, and our code repository is publicly accessible. These efforts allow our experiments to be replicated across different devices without compromising the validity of core conclusions.

❹ **Scope of Evaluated Contexts and Downstream Executability.** Our current implementation and evaluation focus on chat-based PyPI dependency recommendations after `pip install` commands. Therefore, our findings should be interpreted within this monitored setting, and do not yet cover broader dependency specifications such as `requirements.txt`, Dockerfiles, CI configurations, or end-to-end coding-agent workflows. In addition, *PackMonitor* targets package-name validity rather than full program executability, since a valid package may still be API-incompatible, version-sensitive, or environment-dependent. We leave these broader evaluations to future work.

Overall, *PackMonitor* demonstrates that package hallucinations—often regarded as a challenging failure mode of LLMs—can be prevented under well-defined monitoring mechanism. By shifting from post-hoc mitigation to design-level prevention, our approach provides strong package-name validity guarantees guarantees while preserving generation quality and efficiency. At the same time, we acknowledge that these guarantees do not cover all dependency specification contexts or

full program executability, which points to promising directions for future research on trustworthy software generation.

7 Conclusion

In this paper, we propose *PackMonitor*, a novel framework capable of rigorously eliminating package hallucinations in Large Language Models. Operating as a training-free and plug-and-play solution, *PackMonitor* leverages the deterministic nature of package validity to enforce strict compliance during the decoding process. Our extensive experiments demonstrate that *PackMonitor* consistently reduces package hallucination rates to absolute zero across all evaluated models. Crucially, it achieves this strict guarantee with negligible inference overhead and without compromising the models' general code generation capabilities. By closing the gap between generative probability and factual validity, we believe *PackMonitor* paves the way for the trustworthy adoption of LLM in real-world software development.

Data Availability

The source code is provided at: https://github.com/THU-Agent/PackMonitor.

Acknowledgments

This work was supported by Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (No. JYB2025XDXM101), the Research Grant of Tsinghua-Tencent Joint Laboratory for Internet Innovation Technology.

This work was also supported by Tsinghua University - Keystone Electrical (Zhejiang) Co.,Ltd Joint Research Center for Embodied Multimodal Artificial Intelligence (JCEMAI) and the Beijing Natural Science Foundation under Grant No. 4264107.

References

[1] Lakshya A Agrawal, Aditya Kanade, Navin Goyal, Shuvendu Lahiri, and Sriram Rajamani. 2023. Monitor-guided decoding of code lms with static analysis of repository context. *Advances in Neural Information Processing Systems* 36 (2023), 32270–32298.

[2] Iftekhar Ahmed, Aldeida Aleti, Haipeng Cai, Alexander Chatzigeorgiou, Pinjia He, Xing Hu, Mauro Pezzè, Denys Poshyvanyk, and Xin Xia. 2025. Artificial Intelligence for Software Engineering: The Journey So Far and the Road Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 119 (May 2025), 27 pages. doi:10.1145/3719006

[3] Anysphere Inc. 2024. Cursor: The AI-native Code Editor. https://www.cursor.com/. Accessed: 2024-05-20.

[4] Apache Software Foundation. 2004. Apache Maven. https://maven.apache.org/. Last accessed Dec. 2025.

[5] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2024. Guiding llms the right way: Fast, non-invasive constrained generation. *arXiv preprint arXiv:2403.06988* (2024).

[6] Liyi Cai, Yijie Ren, Yitong Zhang, and Jia Li. 2025. AI-Driven Self-Evolving Software: A Promising Path Toward Software Automation. *arXiv preprint arXiv:2510.00591* (2025).

[7] Chong Chen, Jianzhong Su, Jiachi Chen, Yanlin Wang, Tingting Bi, Jianxing Yu, Yanli Wang, Xingwei Lin, Ting Chen, and Zibin Zheng. 2025. When chatgpt meets smart contract vulnerability detection: How far are we? *ACM Transactions on Software Engineering and Methodology* 34, 4 (2025), 1–30.

[8] Haorui Chen, Chengze Li, and Jia Li. 2025. FeatBench: Evaluating Coding Agents on Feature Implementation for Vibe Coding. *arXiv preprint arXiv:2509.22237* (2025).

[9] Junyi Chen, Shihao Bai, Zaijun Wang, Siyu Wu, Chuheng Du, Hailong Yang, Ruihao Gong, Shengzhong Liu, Fan Wu, and Guihai Chen. 2025. Pre^3: Enabling Deterministic Pushdown Automata for Faster Structured LLM Generation. *arXiv preprint arXiv:2506.03887* (2025).

[10] Mark Chen. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[11] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai

Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437

[12] Yanjun Fu, Ethan Baker, Yu Ding, and Yizheng Chen. 2024. Constrained decoding for secure code generation. *arXiv preprint arXiv:2405.00218* (2024).

[13] Cuiyun Gao, Xing Hu, Shan Gao, Xin Xia, and Zhi Jin. 2025. The current challenges of software engineering in the era of large language models. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30.

[14] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* 2, 1 (2023).

[15] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-constrained decoding for structured NLP tasks without finetuning. *arXiv preprint arXiv:2305.13971* (2023).

[16] GitHub Inc. 2024. GitHub Copilot: Your AI pair programmer. https://github.com/features/copilot. Accessed: 2024-05-20.

[17] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence. *ArXiv* abs/2401.14196 (2024). https://api.semanticscholar.org/CorpusID:267211867

[18] Ruida Hu, Chao Peng, Xinchen Wang, Junjielong Xu, and Cuiyun Gao. 2025. Repo2Run: Automated Building Executable Environment for Code Repository at Scale. *arXiv preprint arXiv:2502.13681* (2025).

[19] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. 2025. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Transactions on Information Systems* 43, 2 (Jan. 2025), 1–55. doi:10.1145/3703155

[20] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. Qwen2.5-Coder Technical Report. arXiv:2409.12186 [cs.CL] https://arxiv.org/abs/2409.12186

[21] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM computing surveys* 55, 12 (2023), 1–38.

[22] Ziwei Ji, Tiezheng Yu, Yan Xu, Nayeon Lee, Etsuko Ishii, and Pascale Fung. 2023. Towards mitigating LLM hallucination via self reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 1827–1843.

[23] Arjun Krishna, Erick Galinkin, Leon Derczynski, and Jeffrey Martin. 2025. Importing Phantoms: Measuring LLM Package Hallucination Vulnerabilities. arXiv:2501.19012 [cs.LG] https://arxiv.org/abs/2501.19012

[24] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. Sok: Taxonomy of attacks on open-source software supply chains. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1509–1526.

[25] Ben Lanyado. 2024. Diving Deeper into AI Package Hallucinations. https://www.lasso.security/blog/ai-package-hallucinations/. Last accessed Mar. 2025.

[26] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[27] Chengze Li, Yitong Zhang, Jia Li, Liyi Cai, and Ge Li. 2025. Beyond autoregression: An empirical study of diffusion large language models for code generation. *arXiv preprint arXiv:2509.11252* (2025).

[28] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2025. Structured chain-of-thought prompting for code generation. *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), 1–23.
[29] Jenny T Liang, Chenyang Yang, and Brad A Myers. 2024. A large-scale survey on the usability of ai programming assistants: Successes and challenges. In *Proceedings of the 46th IEEE/ACM international conference on software engineering*. 1–13.
[30] Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, and Li Zhang. 2024. Exploring and Evaluating Hallucinations in LLM-Powered Code Generation. *ArXiv* abs/2404.00971 (2024). https://api.semanticscholar.org/CorpusID:268819908
[31] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems* 36 (2023), 46534–46594.
[32] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. 2020. On faithfulness and factuality in abstractive summarization. *arXiv preprint arXiv:2005.00661* (2020).
[33] Daniel Melcer, Nathan Fulton, Sanjay Krishna Gouda, and Haifeng Qian. 2024. Constrained decoding for code language models via efficient left and right quotienting of context-sensitive grammars. *CoRR* (2024).
[34] Meta. 2024. Introducing Meta Llama 3: The Most Capable Openly Available LLM to Date. https://ai.meta.com/blog/meta-llama-3/. Last accessed Dec. 2025.
[35] Meta. 2024. Llama 3.1. https://ai.meta.com/blog/meta-llama-3-1/. Last accessed Dec. 2025.
[36] Shradha Neupane, Grant Holmes, Elizabeth Wyss, Drew Davidson, and Lorenzo De Carli. 2023. Beyond typosquatting: an in-depth look at package confusion. In *32nd USENIX Security Symposium (USENIX Security 23)*. 3439–3456.
[37] npm, Inc. 2024. npm. https://www.npmjs.com/. Last accessed Dec. 2025.
[38] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020. Backstabber's knife collection: A review of open source software supply chain attacks. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 23–43.
[39] Python Software Foundation. 2000. PyPI. https://pypi.org/. Last accessed Dec. 2025.
[40] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 [cs.CL] https://arxiv.org/abs/2308.12950
[41] Joseph Spracklen, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, and Bimal Viswanath. 2025. We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs. In *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, Seattle, WA, 3687–3706. https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen
[42] Stack Overflow, Inc. 2024. Stack Overflow. https://stackoverflow.com/. Last accessed Dec. 2025.
[43] Katherine Tian, Eric Mitchell, Huaxiu Yao, Christopher D Manning, and Chelsea Finn. 2023. Fine-tuning language models for factuality. In *The Twelfth International Conference on Learning Representations*.
[44] SMTI Tonmoy, SM Zaman, Vinija Jain, Anku Rani, Vipula Rawte, Aman Chadha, and Amitava Das. 2024. A comprehensive survey of hallucination mitigation techniques in large language models. *arXiv preprint arXiv:2401.01313* 6 (2024).
[45] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. arXiv:2407.16741 [cs.SE] https://arxiv.org/abs/2407.16741
[46] Zhuoer Wang, Leonardo FR Ribeiro, Alexandros Papangelis, Rohan Mukherjee, Tzu-Yen Wang, Xinyan Zhao, Arijit Biswas, James Caverlee, and Angeliki Metallinou. 2024. FANTAstic SEquences and where to find them: Faithful and efficient API call generation through state-tracked constrained decoding and reranking. *arXiv preprint arXiv:2407.13945* (2024).
[47] Weibin Wu, Yuhang Cao, Ning Yi, Rongyi Ou, and Zibin Zheng. 2025. Detecting and Reducing the Factual Hallucinations of Large Language Models with Metamorphic Testing. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 1432–1453.
[48] Lekang Yang, Yuetong Liu, Yitong Zhang, and Jia Li. 2025. DiffTester: Accelerating Unit Test Generation for Diffusion LLMs via Repetitive Pattern. *arXiv preprint arXiv:2509.24975* (2025).
[49] Xiao Yu, Lei Liu, Xing Hu, Jacky Wai Keung, Jin Liu, and Xin Xia. 2024. Fight fire with fire: How much can we trust chatgpt on source code-related tasks? *IEEE Transactions on Software Engineering* (2024).
[50] Yitong Zhang, Chengze Li, Ruize Chen, Guowei Yang, Xiaoran Jia, Yijie Ren, and Jia Li. 2026. To see is not to master: Teaching llms to use private libraries for code generation. *arXiv preprint arXiv:2603.15159* (2026).

[51] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Chen Xu, Yulong Chen, Longyue Wang, Anh Tuan Luu, Wei Bi, Freda Shi, and Shuming Shi. 2025. Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models. arXiv:2309.01219 [cs.CL] https://arxiv.org/abs/2309.01219

[52] Yitong Zhang, Yongmin Li, Yuetong Liu, Jia Li, Xiaoran Jia, Zherui Li, and Ge Li. 2026. Lookahead-then-Verify: Reliable Constrained Decoding for Diffusion LLMs under Context-Free Grammars. *arXiv preprint arXiv:2602.00612* (2026).

[53] Yitong Zhang, Shiteng Lu, and Jia Li. 2026. Grammar-Constrained Decoding Can Jailbreak LLMs into Generating Malicious Code. *arXiv preprint arXiv:2606.11817* (2026).

[54] Xiaoqi Zhao, Haipeng Qu, Jianliang Xu, Xiaohui Li, Wenjie Lv, and Gai-Ge Wang. 2024. A systematic review of fuzzing. *Soft Computing* 28, 6 (2024), 5493–5522.

[55] Yukai Zhao, Menghan Wu, Xing Hu, and Xin Xia. 2025. HFuzzer: Testing Large Language Models for Package Hallucinations via Phrase-based Fuzzing. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2746–2758. doi:10.1109/ase63991.2025.00225

Received 2026-01-30; accepted 2026-04-16